C PROGRAMMING — BEGINNER GUIDE

How Casting Really Works

Why a 4-byte int does NOT get squeezed into a 1-byte char — and what actually happens when you cast in C.

You are not moving data.
You are changing
how you look at it.

Open in browser → Ctrl+P → Save as PDF

CHAPTER 01

Start Here — A Real World Story

Before touching any code, let's understand this with something you already know.

ANALOGY

Imagine a building with 4 floors

There is a building with 4 floors. Each floor has 1 room. The building is your **int** — it takes 4 floors of space.

Now you want to look at it floor by floor. You take a small window (a **char pointer**) and look through floor 1, then floor 2, then floor 3, then floor 4.

Did the building change? **No.**
Did floors get removed? **No.**
You just looked at it one floor at a time using a smaller window.

ANALOGY 2

A book with 4 pages

You have a book with 4 pages (your int = 4 bytes).
Reading it normally = open all 4 pages at once = **int pointer**.
Reading it one page at a time = turn page by page = **char pointer**.

The book does not lose pages. All 4 pages are always there. You just chose how many pages to look at each time.

Casting a pointer does NOT move or destroy data.

It only changes your window size.

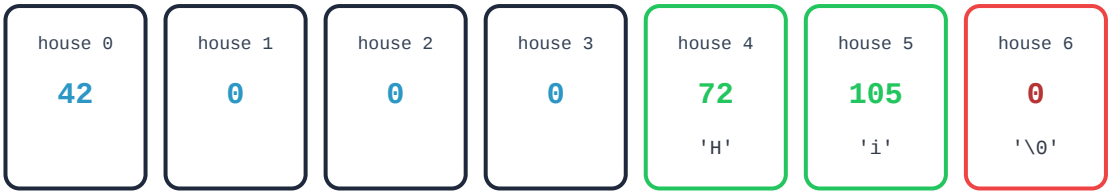
CHAPTER 02

Memory is a Street of Houses

The simplest mental model for understanding what memory looks like.

1 Each house = 1 byte

MEMORY STREET — each box is 1 byte (1 house)



Blue houses (0–3) = the integer 42 stored as 4 bytes | Green (4–5) = string "Hi" | Red (6) = string terminator

2 A pointer is just a house number

What a pointer really is

A pointer stores a house number (address). It tells you WHERE something lives on the memory street. It does NOT contain the data itself — it just says "go to house number X".

```
int n = 42;

// &n  means "what house number does n live at?"
// Let's say the answer is house 0

int  *big_window = &n;   // look at 4 houses at once (int = 4 bytes)
char *small_window = (char *)&n;  // look at 1 house at a time
```

```
// Both point to house 0.
// Neither moved the data.
// Only the window size changed.
```

3 Different window sizes

```
int n = 42; — lives at address 0x100
```

0x100	0x101	0x102	0x103
42	0	0	0
byte 0	byte 1	byte 2	byte 3

```
int *ptr → reads ALL 4 yellow houses together → sees 42
char *ptr → reads 1 house at a time → sees 42, then 0, then 0, then 0
```

✓ **Nothing was destroyed or lost**

All 4 bytes of the int are still there. The char pointer just visits them one at a time instead of reading all 4 at once. This is the whole trick.

CHAPTER 03

Two Kinds of Casting — Very Different

This is the confusion. Casting a VALUE and casting a POINTER are completely different things. Learn to tell them apart.

❌ Casting a VALUE

You take the number itself and try to put it into a smaller box. Data gets cut off. This is dangerous and causes data loss.

✅ Casting a POINTER

You change the window size you use to look at memory. No data moves. No data is lost. The memory stays exactly the same.

1 Casting a VALUE — Data is Lost

Think of it like this

You have a 4-digit number: **1000**. Someone says "put it in a 1-digit box". Only the last digit fits: **0**. The other digits are gone forever. This is value casting.

```
int n = 1000; → what does 1000 look like in 4 bytes?
```

byte 0	byte 1	byte 2	byte 3
232	3	0	0
lowest			highest

```
char c = (char)n; → only byte 0 (232) survives. Bytes 1,2,3 are THROWN AWAY.
```

```
int  n = 1000;
char c = (char)n;   // VALUE cast — dangerous!

// 1000 needs 2 bytes to store properly
// char only has 1 byte (0–255)
// So only the lowest byte survives: 232
// The rest is GONE — this is data loss

printf("%d\n", c);  // prints -24 (not 1000!)
                    // because 232 > 127 → signed char wraps to negative
```

⚠ **Value casting = squeezing = data loss**

When you cast a value to a smaller type, bytes get cut off. Use this only when you KNOW the value fits in the smaller type.

2 Casting a POINTER — Nothing is Lost

Think of it like this

You have the same 4-floor building. You don't move anything. You just change your glasses — one pair shows you all 4 floors at once, another pair shows you one floor at a time. The building is untouched.

```
int  n = 1000;       // n lives somewhere in memory — 4 bytes

// Get a pointer to n
int           *p4 = &n;              // 4-byte window
unsigned char *p1 = (unsigned char *)&n;  // 1-byte window — POINTER cast

// Read through the 4-byte window
*p4;            // reads 4 bytes at once → 1000 ✓

// Read through the 1-byte window, one at a time
```

```
p1[0];          // reads byte 0 → 232
p1[1];          // reads byte 1 → 3
p1[2];          // reads byte 2 → 0
p1[3];          // reads byte 3 → 0

// 232 + 3*256 = 1000 — ALL THE DATA IS STILL THERE
// Nothing was lost. We just read it piece by piece.
```

✓ **Pointer casting = changing glasses = zero data loss**

The int still occupies 4 bytes. The char pointer just looks at those 4 bytes one at a time.

This is exactly how memcpy, memset, and memcmp work.

CHAPTER 04

Visual Proof — See It Happen

Let's trace through exactly what happens in memory, step by step, with a real number.

1 We store the number 12345678 as an int

How does 12345678 look in binary / bytes?

12345678 in binary is: `00000000 10111100 01100001 01001110`

That is 4 groups of 8 bits = 4 bytes. Each byte can be written as a number 0–255.

```
int n = 12345678; stored in memory at address 0x100
```

0x100	0x101	0x102	0x103
78	97	188	0
byte 0	byte 1	byte 2	byte 3

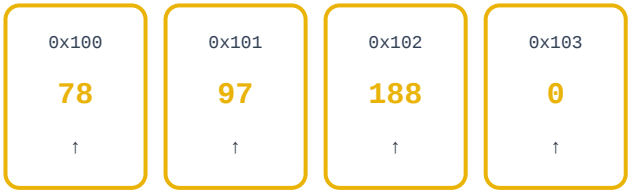
```
All 4 bytes together = 12345678 (little-endian order: lowest byte first)
```

2 int pointer reads all 4 at once

```
int *p4 = &n;

printf("%d\n", *p4);
// Goes to 0x100, reads 4 bytes, combines them → 12345678 ✓
```

```
*p4 — reading with int pointer (4-byte window)
```



← all 4 bytes read together → result: 12345678

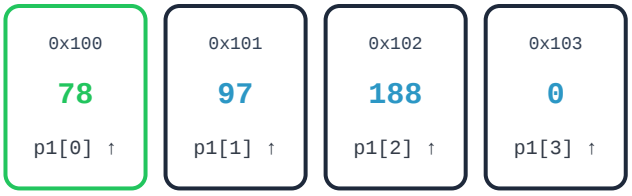
3 char pointer reads 1 byte at a time — same memory

```
unsigned char *p1 = (unsigned char *)&n;  // same address 0x100

printf("%d\n", p1[0]);   // reads 0x100 → 78
printf("%d\n", p1[1]);   // reads 0x101 → 97
printf("%d\n", p1[2]);   // reads 0x102 → 188
printf("%d\n", p1[3]);   // reads 0x103 → 0

// All 4 bytes are still there — we just read them one by one
```

p1[0], p1[1], p1[2], p1[3] — reading with char pointer (1-byte window each time)



Each read is 1 byte — but all 4 bytes are still there, unchanged

The int was never inside the char.

The char pointer just visited the int's bytes.

CHAPTER 05

How ft_memcpy Uses This Trick

Now you understand the trick — let's see how it makes memcpy work for ANY type without knowing the type.

1 The problem memcpy solves

How do you copy data when you don't know its type?

memcpy receives a `void *` — a raw address with no type. It doesn't know if it's an int, a struct, a string, or a float. How can it copy the data correctly?

Answer: **cast to unsigned char *, then copy byte by byte.** It doesn't matter what the type is — ALL data is bytes. Copy the bytes, you copy everything.

2 Step-by-step trace of memcpy copying an int

```
int src = 12345678;
int dst = 0;

ft_memcpy(&dst, &src, sizeof(int));  // copy 4 bytes
```

BEFORE — src at 0x100, dst at 0x200

0	0	0	0
empty	empty	empty	empty

1 **Cast both void* to unsigned char***

Now we can walk through memory one byte at a time

2 **Read byte 0 from src (78), write to dst byte 0**

dst[0] = src[0] = 78

3 **Read byte 1 from src (97), write to dst byte 1**

dst[1] = src[1] = 97

4 **Read byte 2 from src (188), write to dst byte 2**

dst[2] = src[2] = 188

5 **Read byte 3 from src (0), write to dst byte 3**

dst[3] = src[3] = 0 — all 4 bytes copied

AFTER — dst now has a perfect copy

src 0x100	src 0x101	src 0x102	src 0x103
78	97	188	0
unchanged	unchanged	unchanged	unchanged

dst 0x200	dst 0x201	dst 0x202	dst 0x203
78	97	188	0
copied ✓	copied ✓	copied ✓	copied ✓

3 The actual code — beautifully simple

```
void *ft_memcpy(void *dst, const void *src, size_t n)
{
    unsigned char       *d = (unsigned char       *)dst;
    const unsigned char *s = (const unsigned char *)src;
    // ↑ cast pointer — no data moved, just window size = 1 byte

    while (n--)         // repeat n times (n = 4 for an int)
        *d++ = *s++;    // copy 1 byte, move both pointers 1 step forward

    return (dst);
}
```

✓ **memcpy works for ALL types because of this**

It doesn't matter if you're copying an int, a struct, a float, or a string. Everything is bytes. Copy the bytes, you copy everything. Zero data loss.

CHAPTER 06

Side by Side — Value vs Pointer Cast

The clearest comparison to keep in your head.

Question	Value Cast (char)n	Pointer Cast (char *)&n
`What moves?`	The number itself	Only the address (pointer value)
`Data lost?`	Yes — bytes that don't fit are cut	No — all bytes stay in memory
`Original data changed?`	No (the int stays)	No (the int stays)
`Memory changes?`	No	No
`What changes?`	The value stored in char c	The window size used to read memory
`When to use?`	When you know value fits in smaller type	When doing byte-by-byte memory work
`Used in libft?`	Rarely (only for single char writes)	Yes — memset, memcpy, memcmp, memchr

1 Code comparison

❌ Value Cast — data cut

```
int  n = 1000;
char c = (char)n;
// c = -24 (not 1000!)
// 3 bytes were CUT
```

✅ Pointer Cast — nothing cut

```
int  n = 1000;
unsigned char *p = (unsigned char *)&n;
// p[0]+p[1]+p[2]+p[3] = 1000
// all bytes still there
```

CHAPTER 07

Why unsigned char — Not Just char

One last concept — why we always use unsigned char for byte work, not just char.

ANALOGY

A ruler that starts at -128 vs a ruler that starts at 0

Signed char is like a ruler that goes from -128 to 127.
Unsigned char is like a ruler that goes from 0 to 255.

A byte can hold values 0 to 255. If you use a signed ruler, any value above 127 becomes negative — which breaks comparisons and arithmetic.

Use the ruler that matches the range. For bytes: always unsigned char.

Same byte — two different interpretations

byte
200
11001000

char (signed):	unsigned char:
200 > 127 → wraps to -56	200 stays as 200
comparisons break!	always correct ✓

```
// Bug — signed char comparison
char a = 200;   // stored as -56 (signed overflow!)
char b = 100;
```

```
a - b;          // -56 - 100 = -156  ← WRONG

// Fix — unsigned char
(unsigned char)a - (unsigned char)b;
// 200 - 100 = 100  ← CORRECT
```

⚠ **The rule for all memory functions in libft**

Whenever you work with raw bytes — comparisons, writing, reading — always cast to `unsigned char *` first. This applies to memset, memcpy, memmove, memchr, memcmp, strchr, strncmp.

CHAPTER 08

Everything Together

The complete picture in one place.

An int is 4 bytes.
A char pointer visits those 4 bytes one at a time.

The int never goes inside the char.

1 The 3 things to always remember

1 **Casting a value = squeezing = data loss**

`(char)my_int` — takes the number itself and cuts it to fit. Bytes are lost. Only use when value fits.

2 **Casting a pointer = new glasses = zero data loss**

`(char *)&my_int` — changes the window size. Nothing moves. All bytes stay. You just read them differently.

3 **Always use unsigned char for byte work**

Bytes are 0–255. Signed char goes to -128 which breaks math. unsigned char matches the actual byte range perfectly.

2 Final cheat sheet

You see in code	What it means	Data lost?
`(char)n`	Take the number, cut it to 1 byte	YES if n > 127
`(char *)&n`	Look at n's memory 1 byte at a time	NO
`(unsigned char *)ptr`	Look at memory, treat each byte as 0–255	NO
`(void *)ptr`	Just store the address, no type claim	NO
`(int *)void_ptr`	Read 4 bytes at once from this address	NO
`ptr++`	Move to next element (size depends on type)	NO

THE FINAL ANALOGY

A pizza with 4 slices

Your int is a pizza with 4 slices (4 bytes).

Value cast = you tried to fit the whole pizza in your mouth at once. You can only fit 1 slice. The other 3 fall on the floor. They are gone.

Pointer cast = you eat 1 slice at a time. The pizza stays on the table. After 4 trips, you've eaten all 4 slices. Nothing was lost — you just changed how you ate it.

The char pointer eats the int 1 slice (byte) at a time.

You had the right instinct.
Everything IS bytes.

Casting just changes how you read them.